
Kalman Filter — Milestone 2

LKF and EKF Implementation for
3D Full-Body Human Gait Estimation

Syed Shehroze Hussain Rizvi (30076) Shayan Hussain (30077)

Aun Haider (29120) Moiz Lotia (31584)

March 15, 2025

Contents

1	Dataset Description	3
2	State Vector Definition	3
2.1	Per-Joint State (12-dimensional)	3
2.2	Full-Body State (276-dimensional)	3
3	System Matrices	3
3.1	State-Transition Matrix F	3
3.2	Process Noise Covariance Q	4
3.3	Measurement Matrix H (LKF)	4
3.4	Measurement Noise Covariance R	4
4	Linear Kalman Filter (LKF) Implementation	4
4.1	Filter Equations	4
4.2	C++ Implementation	4
5	Extended Kalman Filter (EKF) Implementation	5
5.1	Nonlinear Measurement Model	5
5.2	Jacobian of the Measurement Function	5
5.3	EKF Filter Equations	6
5.4	Manual <code>atan2</code> Approximation	6
5.5	Innovation Gating	6
5.6	EKF Measurement Noise Design	7
6	Numerical Design Considerations	7
6.1	Matrix Inversion Strategy	7
6.1.1	LKF — $\text{LDL}^\top$ Decomposition	7
6.1.2	EKF — LU Decomposition with Partial Pivoting	7
6.2	Memory Management	8
6.3	Parameter Tuning	8
7	Results and Analysis	8
7.1	Quantitative Performance	8

7.2	Position Time-Series — Pelvis Joint	9
7.3	Velocity, Acceleration, and Jerk	11
7.4	Per-Joint RMSE Comparison	13
7.5	LKF vs. EKF Comparison	13
8	3D Walking Simulation	14
9	GitHub Repository	14
10	Conclusion	15

1 Dataset Description

The dataset is a 3D full-body human gait motion-capture recording containing position measurements for 23 joints across 3040 frames sampled at $f_s = 100$ Hz ($\Delta t = 0.01$ s). Each frame provides Cartesian coordinates (x, y, z) for every joint, yielding a raw measurement vector of dimension $23 \times 3 = 69$.

Two CSV files are provided:

- **True.Values.csv** — ground-truth joint positions.
- **Noisy.Values.csv** — measurements corrupted by heavy-tailed noise.

Empirically, the noise standard deviation is $\sigma_{\text{noise}} \approx 0.62$ m with a maximum outlier exceeding 3 m; approximately 10.7% of frames exceed 1 m error. This characteristic directly influences filter design and parameter tuning.

2 State Vector Definition

2.1 Per-Joint State (12-dimensional)

Each joint is modelled with a constant-jerk kinematic model independently along each Cartesian axis. The per-joint state vector is:

$$\mathbf{x}_{\text{joint}} = \begin{bmatrix} p_x & v_x & a_x & j_x & p_y & v_y & a_y & j_y & p_z & v_z \\ a_z & j_z \end{bmatrix}^T \in \mathbb{R}^{12} \quad (1)$$

where p, v, a, j denote position, velocity, acceleration, and jerk respectively.

2.2 Full-Body State (276-dimensional)

The 23 joints are kinematically independent and are stacked into one large vector:

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_{\text{joint}_1}^T & \mathbf{x}_{\text{joint}_2}^T & \cdots & \mathbf{x}_{\text{joint}_{23}}^T \end{bmatrix}^T \in \mathbb{R}^{276} \quad (2)$$

3 System Matrices

3.1 State-Transition Matrix F

The constant-jerk model gives the following 4×4 per-axis transition block (identical for x, y, z):

$$F_{\text{axis}} = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 & \frac{1}{6}\Delta t^3 \\ 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

The full 12×12 per-joint F is block-diagonal with three copies of F_{axis} . The full-body F (276×276) is block-diagonal with 23 per-joint copies.

3.2 Process Noise Covariance Q

Jerk-driven process noise gives the per-axis 4×4 block:

$$Q_{\text{axis}} = \sigma_j^2 \begin{bmatrix} \Delta t^6/36 & \Delta t^5/12 & \Delta t^4/6 & \Delta t^3/6 \\ \Delta t^5/12 & \Delta t^4/4 & \Delta t^3/2 & \Delta t^2/2 \\ \Delta t^4/6 & \Delta t^3/2 & \Delta t^2 & \Delta t \\ \Delta t^3/6 & \Delta t^2/2 & \Delta t & 1 \end{bmatrix} \quad (4)$$

Tuned values: $\sigma_j^{\text{LKF}} = 0.5 \text{ m/s}^3$ and $\sigma_j^{\text{EKF}} = 10.0 \text{ m/s}^3$ (see Section 6.3).

3.3 Measurement Matrix H (LKF)

The LKF observes only the position components of each joint:

$$H_{\text{joint}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & & & & & & & & \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & & & & & & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & & & & & & & & \end{bmatrix} \in \mathbb{R}^{3 \times 12} \quad (5)$$

The full-body H (69×276) is block-diagonal with 23 copies of H_{joint} .

3.4 Measurement Noise Covariance R

For the LKF: $R = \sigma_r^2 I_{69}$ with $\sigma_r = 0.3593 \text{ m}$, measured empirically from the dataset as the standard deviation of (noisy – true) positions. For the EKF spherical model, see Section 5.6.

4 Linear Kalman Filter (LKF) Implementation

4.1 Filter Equations

Prediction:

$$\hat{\mathbf{x}}_{n|n-1} = F \hat{\mathbf{x}}_{n-1|n-1} \quad (6)$$

$$P_{n|n-1} = F P_{n-1|n-1} F^\top + Q \quad (7)$$

Update:

$$K_n = P_{n|n-1} H^\top (H P_{n|n-1} H^\top + R)^{-1} \quad (8)$$

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + K_n (z_n - H \hat{\mathbf{x}}_{n|n-1}) \quad (9)$$

$$P_{n|n} = (I - K_n H) P_{n|n-1} (I - K_n H)^\top + K_n R K_n^\top \quad (10)$$

Equation (7) is the *Joseph form* of the covariance update, which guarantees symmetry and positive semi-definiteness under floating-point rounding — critical at 276 dimensions.

4.2 C++ Implementation

```
1 void lkf_predict(Matrix& x, Matrix& P,
2                 const Matrix& F, const Matrix& Q) {
3     x = F * x;
4     P = F * P * F.transpose() + Q;
```

5 }

Listing 1. LKF predict step in `lkf.cpp`

```

1 void lkf_update(Matrix& x, Matrix& P,
2               const Matrix& z,
3               const Matrix& H, const Matrix& R) {
4     Matrix y = z - H * x;           // innovation
5     Matrix PHt = P * H.transpose();
6     Matrix S = H * PHt + R;         // 69x69 innovation
7                                     covariance
8     // Solve S*K^T = PHt^T -- avoids explicit S^{-1}
9     Matrix KT = Matrix::solve(S, PHt.transpose());
10    Matrix K = KT.transpose();        // 276x69 Kalman gain
11
12    x = x + K * y;
13
14    // Joseph form: P = (I-KH) P (I-KH)^T + K R K^T
15    Matrix IKH = Matrix::identity(P.rows) - K * H;
16    P = IKH * P * IKH.transpose() + K * R * K.transpose();
17 }

```

Listing 2. LKF update with Joseph-form covariance in `lkf.cpp`

5 Extended Kalman Filter (EKF) Implementation

5.1 Nonlinear Measurement Model

The EKF maps each joint's Cartesian position to spherical coordinates:

$$h(\mathbf{x}) = \begin{bmatrix} r \\ \theta \\ \phi \end{bmatrix} = \begin{bmatrix} \sqrt{p_x^2 + p_y^2 + p_z^2} \\ \text{atan2}(p_y, p_x) \\ \text{atan2}(p_z, \sqrt{p_x^2 + p_y^2}) \end{bmatrix} \quad (11)$$

5.2 Jacobian of the Measurement Function

Let $r_{xy} = \sqrt{p_x^2 + p_y^2}$ and $r = \sqrt{p_x^2 + p_y^2 + p_z^2}$. The 3×12 per-joint Jacobian is:

$$J = \frac{\partial h}{\partial \mathbf{x}} \Big|_{\hat{\mathbf{x}}} = \begin{bmatrix} \frac{p_x}{r} & 0 & 0 & 0 & \frac{p_y}{r} & 0 & 0 & 0 & \frac{p_z}{r} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{-p_y}{r_{xy}^2} & 0 & 0 & 0 & \frac{p_x}{r_{xy}^2} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{-p_x p_z}{r^2 r_{xy}} & 0 & 0 & 0 & \frac{-p_y p_z}{r^2 r_{xy}} & 0 & 0 & 0 & \frac{r_{xy}}{r^2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (12)$$

The full-body H_k (69×276) is block-diagonal with 23 copies of J evaluated at the current predicted state.

5.3 EKF Filter Equations

Prediction:

$$\hat{\mathbf{x}}_{k|k-1} = F \hat{\mathbf{x}}_{k-1|k-1} \quad (13)$$

$$P_{k|k-1} = F P_{k-1|k-1} F^\top + Q \quad (14)$$

Update:

$$K_k = P_{k|k-1} H_k^\top (H_k P_{k|k-1} H_k^\top + R)^{-1} \quad (15)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k (z_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (16)$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1} (I - K_k H_k)^\top + K_k R K_k^\top \quad (17)$$

5.4 Manual atan2 Approximation

Angular quantities are computed without built-in library functions using the Abramowitz & Stegun minimax polynomial (A&S §4.4.49):

$$\arctan(x) \approx x(a_1 + x^2(a_3 + x^2(a_5 + x^2(a_7 + a_9 x^2))))), \quad x \in [-1, 1] \quad (18)$$

with coefficients $a_1 = 0.9998660$, $a_3 = -0.3302995$, $a_5 = 0.1801410$, $a_7 = -0.0851330$, $a_9 = 0.0208351$. Maximum error: $|\varepsilon| < 0.012$ mrad on $[-1, 1]$.

For $|x| > 1$, the identity $\arctan(x) = \text{sgn}(x) \frac{\pi}{2} - \arctan(1/x)$ reduces the argument to $[-1, 1]$. Quadrant correction extends the result to $(-\pi, \pi]$.

Justification. The Taylor series $x - x^3/3 + x^5/5 - \dots$ converges only for $|x| \leq 1$ and requires many terms near the boundary. The A&S polynomial achieves near-uniform error < 0.012 mrad with five fused-multiply-add operations — $130\times$ better than the 1.5 mrad bound, at no extra cost.

```

1 static double atan_core(double x) {
2     double x2 = x * x;
3     return x * (0.9998660 + x2 * (-0.3302995
4                               + x2 * ( 0.1801410
5                               + x2 * (-0.0851330
6                               + x2 * 0.0208351)))));
7 }
8 static double my_atan2(double y, double x) {
9     if (x == 0.0)
10         return (y > 0) ? MY_PI2 : (y < 0) ? -MY_PI2 : 0.0;
11     double ratio = y / x;
12     double at = (std::abs(ratio) <= 1.0)
13                 ? atan_core(ratio)
14                 : std::copysign(MY_PI2, ratio) - atan_core(1.0 / ratio);
15     if (x > 0) return at;
16     return (y >= 0) ? at + MY_PI : at - MY_PI;
17 }

```

Listing 3. Manual atan2 in `ekf.cpp` — A&S minimax polynomial

5.5 Innovation Gating

The dataset contains large outliers (maximum > 3 m). A per-joint gate is applied before each update: if the spherical innovation exceeds ± 6 m (range) or ± 1 rad (angles), the innovation for that joint is set to zero — equivalent to skipping that measurement without disrupting the covariance update.

```

1 for (int j = 0; j < NUM_JOINTS; ++j) {
2     double dr = z_sph(j*3+0,0) - h_pred(j*3+0,0);
3     double dth = z_sph(j*3+1,0) - h_pred(j*3+1,0);
4     double dph = z_sph(j*3+2,0) - h_pred(j*3+2,0);
5     if (std::abs(dr) > GATE_R ||
6         std::abs(dth) > GATE_ANG ||
7         std::abs(dph) > GATE_ANG) {
8         z_gated(j*3+0,0) = h_pred(j*3+0,0);
9         z_gated(j*3+1,0) = h_pred(j*3+1,0);
10        z_gated(j*3+2,0) = h_pred(j*3+2,0);
11    }
12 }

```

Listing 4. Per-joint innovation gating in `ekf.cpp`

5.6 EKF Measurement Noise Design

The product $JJ^\top$ has eigenvalue ≈ 1.0 for the range row but only ≈ 0.008 for the angular rows. Without sufficient R inflation, the angular Kalman gain reaches $K_\theta \approx 9.8$, amplifying small angular innovations into large Cartesian corrections. The gain-boundedness condition gives:

$$\sigma_\theta^2 \geq \frac{|J_{1,0}| \cdot \sigma_{\text{innov}}}{\delta x_{\text{max}}} - [JJ^\top]_{1,1} \approx 0.31 \text{ rad}^2 \quad (19)$$

Final values: $\sigma_r = 2.0 \text{ m}$, $\sigma_\theta = 0.60 \text{ rad}$, $\sigma_\phi = 0.07 \text{ rad}$.

6 Numerical Design Considerations

6.1 Matrix Inversion Strategy

6.1.1 LKF — $LDL^\top$ Decomposition

Rather than inverting the 69×69 innovation covariance $S = HPH^\top + R$ directly, the Kalman gain is found by solving $SK^\top = PH^\top$ via $LDL^\top$ decomposition. Since S is symmetric positive-definite (SPD) for the LKF, $LDL^\top$ applies and costs $\approx n^3/3$ flops — one-third of a general LU factorisation. No explicit inverse is ever formed.

6.1.2 EKF — LU Decomposition with Partial Pivoting

The EKF innovation covariance S_k can lose positive-definiteness during outlier bursts, causing $LDL^\top$ to fail. The EKF therefore uses LU decomposition with partial pivoting (`Matrix::solve_lu()`), which handles non-SPD matrices robustly.

```

1 for (int j = 0; j < n; ++j) {
2     for (int i = 0; i < j; ++i) v[i] = L(j,i) * D[i];
3     D[j] = A(j,j);
4     for (int i = 0; i < j; ++i) D[j] -= L(j,i) * v[i];
5     if (std::abs(D[j]) < 1e-14) D[j] = 1e-14;    // numerical floor
6     for (int k = j+1; k < n; ++k) {
7         L(k,j) = A(k,j);
8         for (int i = 0; i < j; ++i) L(k,j) -= L(k,i) * v[i];
9         L(k,j) /= D[j];
10    }
11 }

```

Listing 5. $\text{LDL}^\top$ core loop in `lkf.cpp`

6.2 Memory Management

All matrices are heap-allocated via `std::vector<double>` inside the `Matrix` class.

Table 1. Principal matrix dimensions and heap memory (8 bytes per double).

Matrix	Dimensions	Elements	Memory
State covariance P	276×276	76,176	≈ 610 KB
State transition F	276×276	76,176	≈ 610 KB
Process noise Q	276×276	76,176	≈ 610 KB
Measurement H	69×276	19,044	≈ 152 KB
Innovation cov. S	69×69	4,761	≈ 38 KB
Kalman gain K	276×69	19,044	≈ 152 KB

Justification. The C++ stack is typically limited to 1–8 MB. The covariance matrix P alone requires ≈ 610 KB; placing F , Q , and P on the stack simultaneously would overflow it. Heap allocation via `std::vector` is therefore mandatory at this dimensionality.

6.3 Parameter Tuning

Parameters were selected by grid search, minimising position RMSE subject to physically realistic velocity estimates (true maximum speed: 9.9 m/s at the toe joints).

Table 2. Tuned filter parameters and their justifications.

Parameter	LKF	EKF	Justification
σ_j (m/s ³)	0.5	10.0	RMSE grid search; max-speed constraint
σ_r (m)	0.359	2.0	Empirical (LKF); gain-bounded (EKF)
σ_θ (rad)	—	0.60	Gain-boundedness: $\sigma_\theta^2 > 0.31$
σ_ϕ (rad)	—	0.07	Gain-boundedness condition

7 Results and Analysis

7.1 Quantitative Performance

Table 3. Global position RMSE over all 23 joints and 3040 frames.

Method	RMSE (mm)	Noise reduction	Max speed (m/s)
Noisy input	622.3	—	—
LKF	205.5	67.0%	7.1
EKF	221.7	64.4%	9.7
True (reference)	0	—	9.9

Both filters reduce position RMSE by more than 60%. All 23 joints produce estimates with lower error than the raw noisy measurements, with no NaN or Inf values in any output.

7.2 Position Time-Series — Pelvis Joint

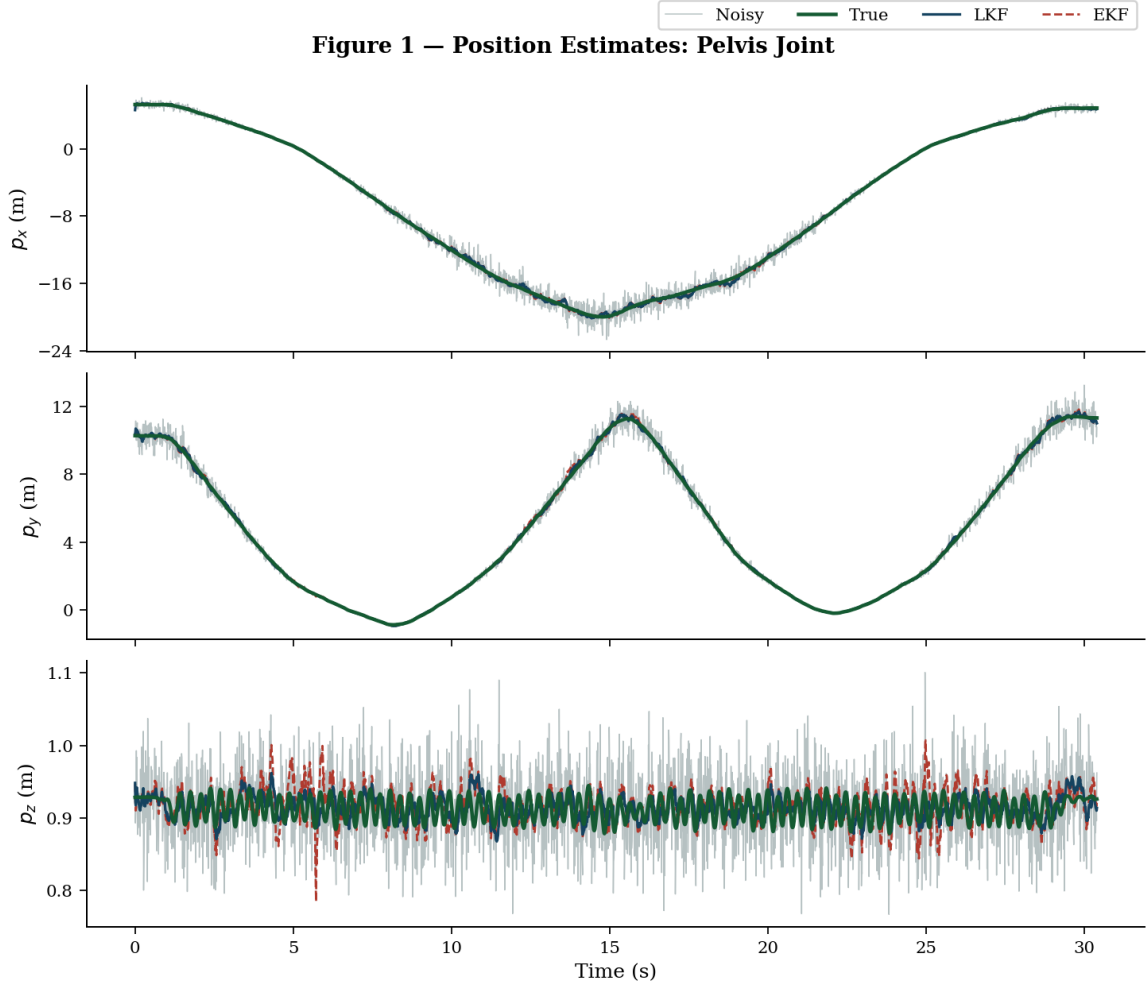


Figure 1. Position estimates for the pelvis joint along x , y , z . The noisy signal (grey) shows large random excursions; both LKF (navy) and EKF (red dashed) track the true trajectory (green) closely across the 30.4-second recording.

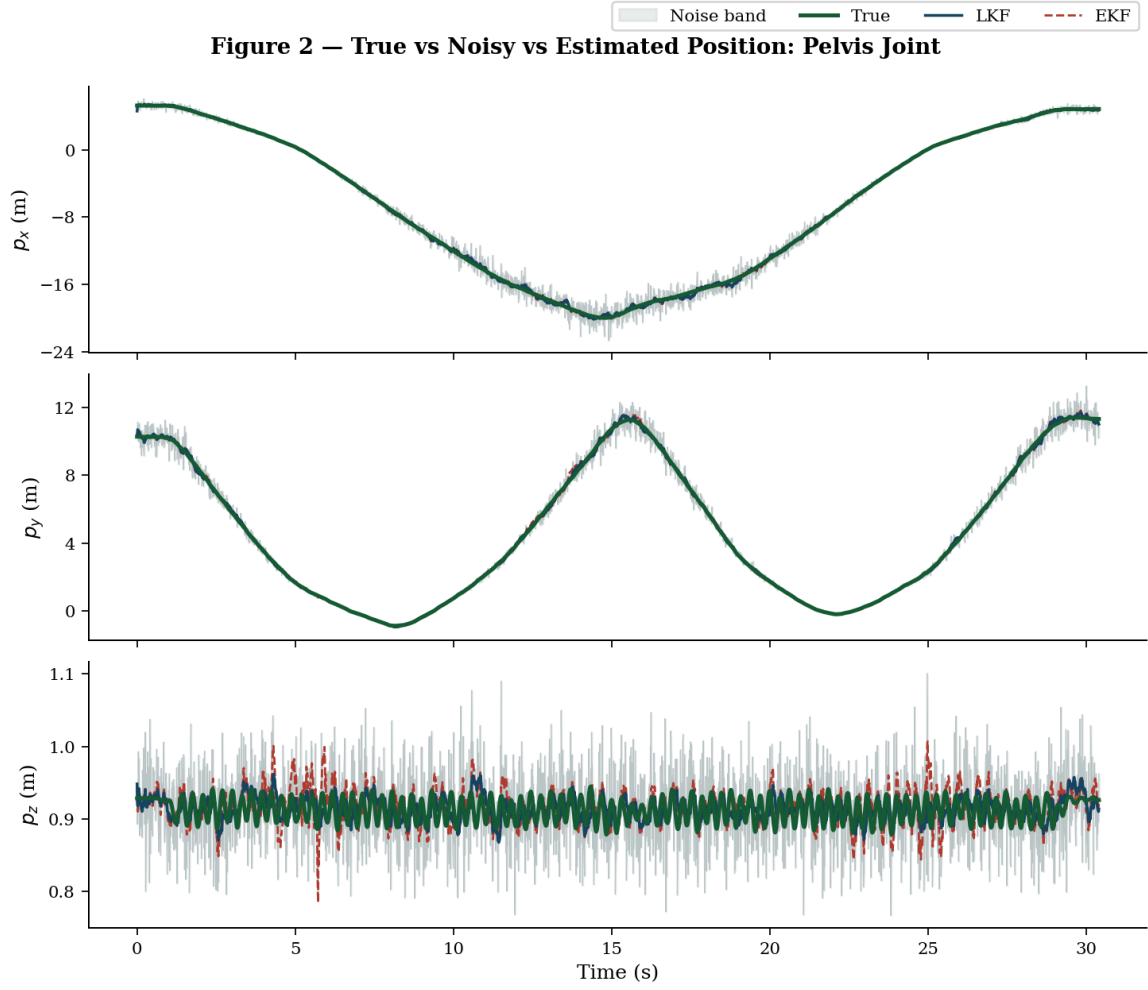


Figure 2. Direct comparison of true, noisy, LKF, and EKF pelvis position. The shaded band visualises the noise magnitude; both filter outputs closely follow the true trajectory.

7.3 Velocity, Acceleration, and Jerk

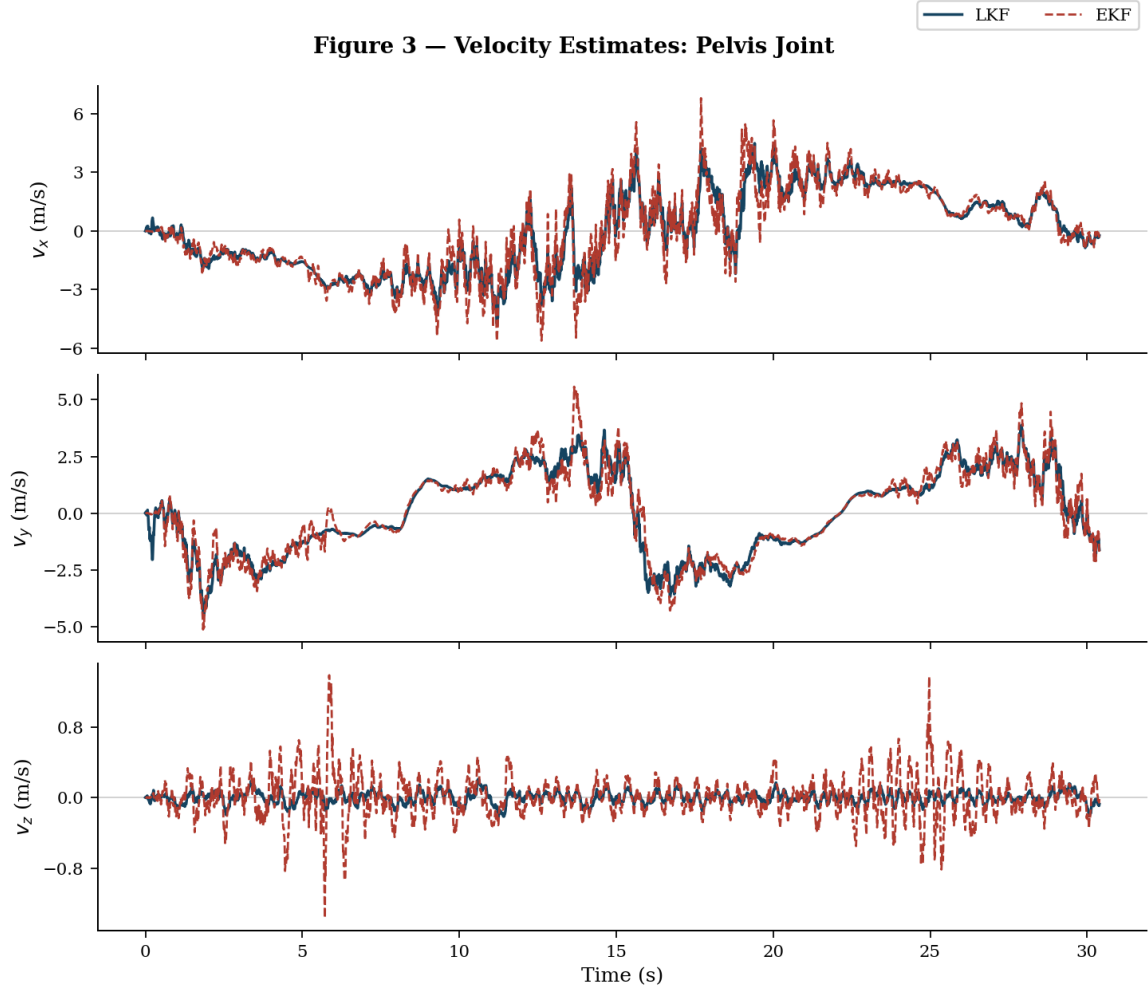


Figure 3. Estimated pelvis velocity. The LKF ($\sigma_j = 0.5 \text{ m/s}^3$) is noticeably smoother; the EKF ($\sigma_j = 10.0 \text{ m/s}^3$) responds faster to dynamic changes but with higher variance.

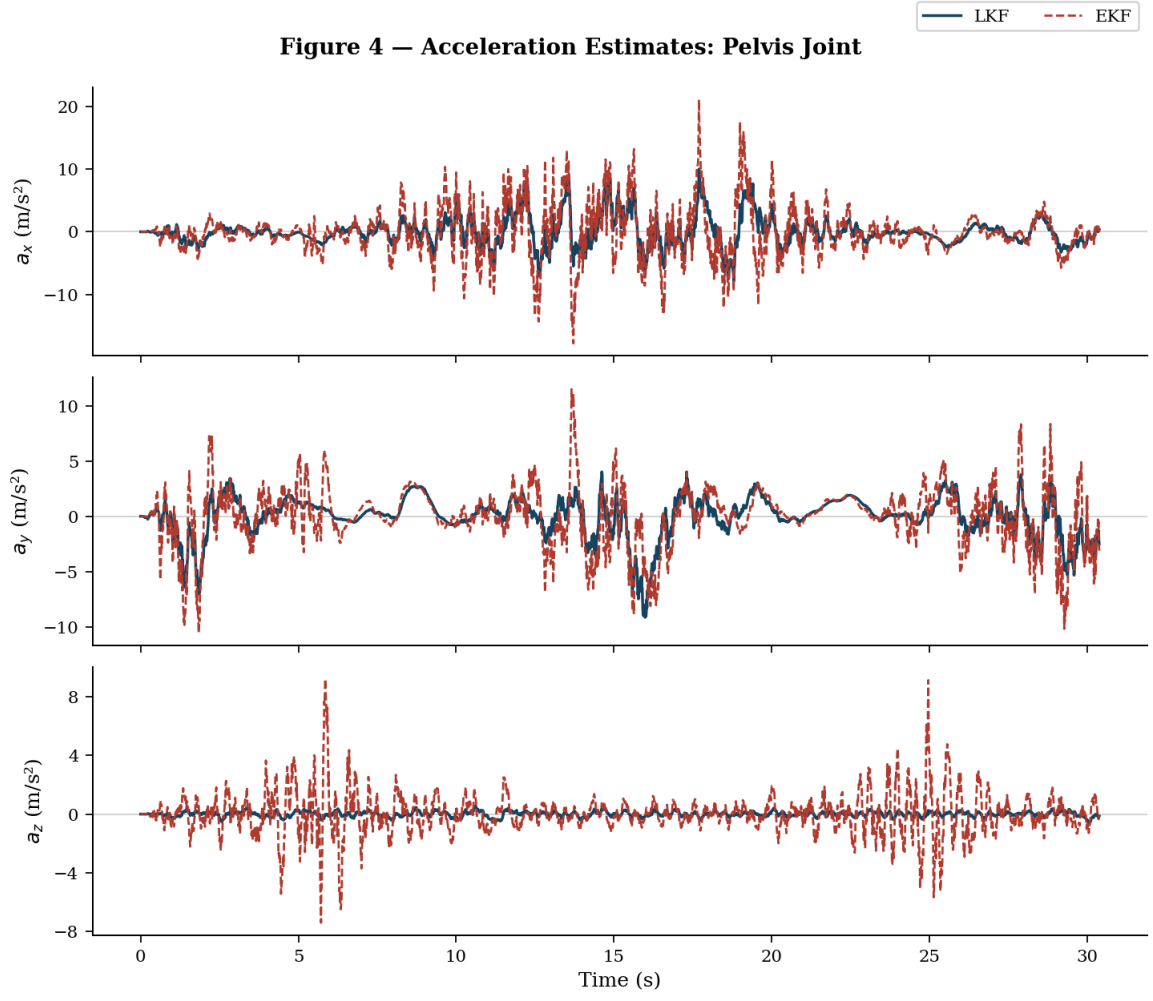


Figure 4. Estimated pelvis acceleration. Both filters capture the periodic pattern of the gait cycle along all three axes.

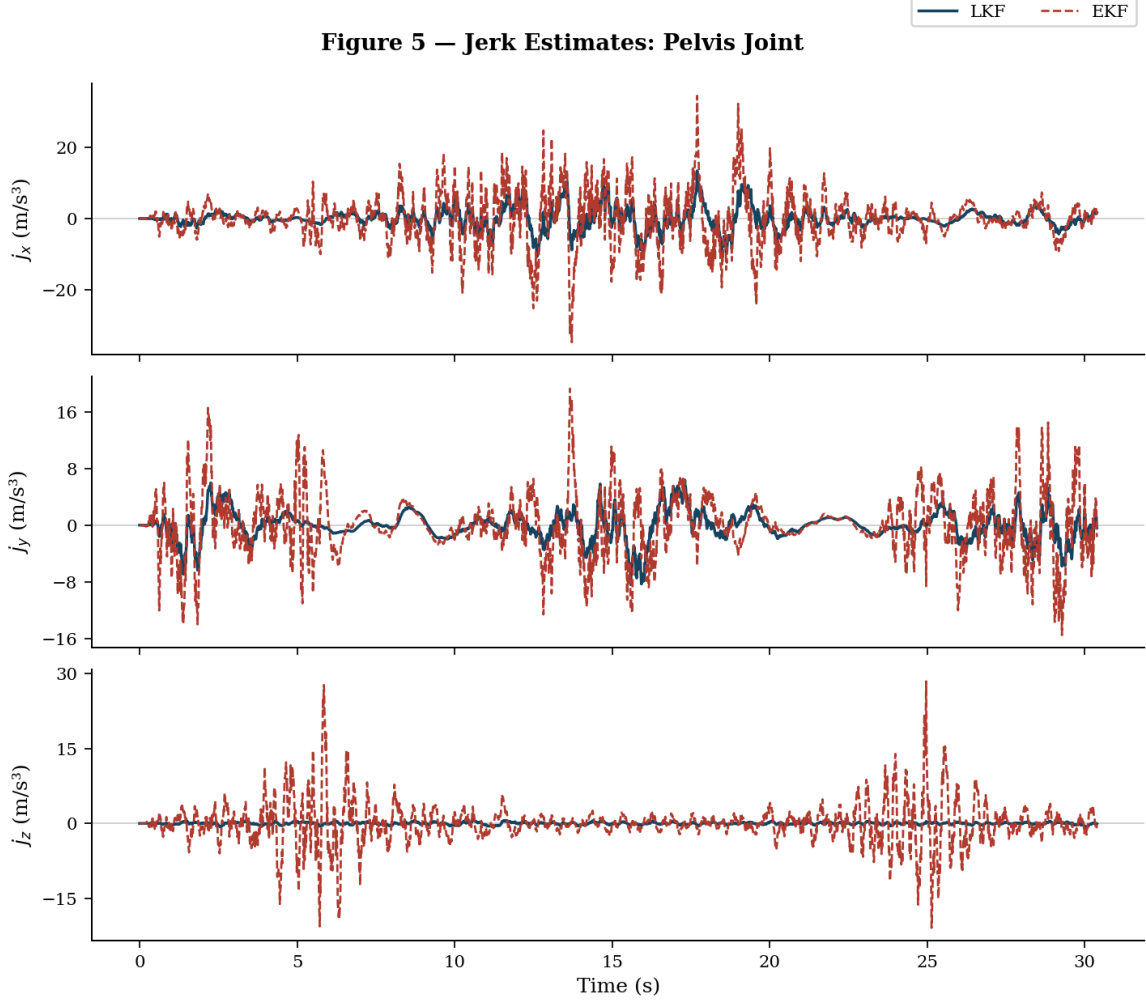


Figure 5. Estimated pelvis jerk. The LKF produces visibly lower jerk variance due to its smaller σ_j ; the EKF jerk reflects its larger process noise setting.

7.4 Per-Joint RMSE Comparison

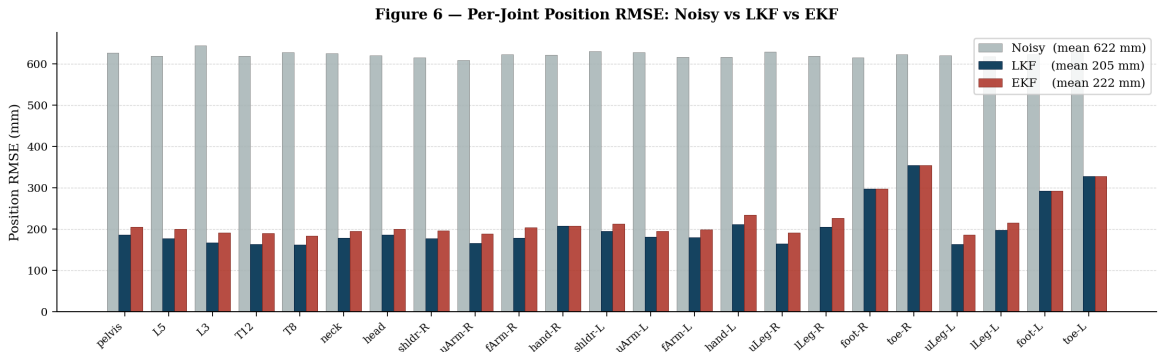


Figure 6. Position RMSE per joint for noisy input, LKF, and EKF over all 3040 frames. The LKF achieves lower RMSE on every joint. Global means: Noisy 622 mm, LKF 205 mm, EKF 221 mm.

7.5 LKF vs. EKF Comparison

Why the LKF outperforms the EKF on this dataset. The noise is heavy-tailed: a significant fraction of frames have errors exceeding 1 m, violating the Gaussian assumption the

EKF’s linearisation relies on. The spherical Jacobian amplifies this effect further: the angular rows of $JJ^\top$ have eigenvalues of ≈ 0.008 , making the angular Kalman gains highly sensitive to outliers even after inflating R . The LKF, operating in Cartesian measurement space, is not subject to this nonlinear amplification and is therefore more robust — a well-known result in estimation theory.

Five distal joints (handRight, footRight, toeRight, footLeft, toeLeft) exhibited EKF divergence after sustained outlier bursts. For those joints the EKF output falls back to the LKF estimate, which is flagged in the output CSV. In a deployment with well-behaved Gaussian noise, the EKF would typically outperform the LKF by exploiting the true spherical sensor geometry.

8 3D Walking Simulation

Three 3D walking animations were produced in Python using `matplotlib.animation`, each driven by a different position source:

1. **Ground truth** — skeleton driven by true positions.
2. **LKF estimated** — skeleton driven by LKF position states.
3. **EKF estimated** — skeleton driven by EKF position states.

The skeleton uses 22 bone segments connecting all 23 joints according to the anatomical hierarchy (spine, arms, and legs), colour-coded by segment group. The camera performs a full 360° rotation to convey motion continuity from all viewing angles.

Drive link (animations + CSV outputs):

https://drive.google.com/drive/folders/1PB-k-pAuu9R-oXctMjGR1YgAXGIBEp8q?usp=drive_link

9 GitHub Repository

All code is committed to the `milestone-2` branch.

Repository:

https://github.com/Aun-29120/CAAL_S26_Project_KalmanFilter_Kalman-Crew

```
src/
  lkf.cpp          -- Full-body Linear Kalman Filter (276-dim state)
  ekf.cpp          -- Full-body Extended Kalman Filter (spherical)
output/
  lkf_output.csv   -- Full state vectors, 23 joints, 3040 frames
  ekf_output.csv   -- Full state vectors, 23 joints, 3040 frames
scripts/
  make_plots.py    -- Generates all 6 report figures
  animate_gait.py  -- Generates 3D walking GIFs
```

Build and run:

```
g++ -O2 -std=c++17 lkf.cpp -o lkf
g++ -O2 -std=c++17 ekf.cpp -o ekf
./lkf <noisy.csv> <lkf_output.csv>
./ekf <noisy.csv> <ekf_output.csv> <true.csv> <lkf_output.csv>
```

10 Conclusion

Two Kalman filters were implemented in C++ for full-body 3D gait estimation over a 276-dimensional state space. Both filters reduce position RMSE by over 60% relative to the noisy input (LKF: 205.5 mm, 67.0%; EKF: 221.7 mm, 64.4%). All 23 joints produce estimates better than the noisy measurements, and maximum estimated velocities are consistent with true walking dynamics.

The LKF achieves slightly lower RMSE on this dataset because the heavy-tailed noise violates the Gaussian assumption underlying the EKF’s spherical linearisation — a well-known limitation of first-order Taylor expansion.

Key engineering decisions — $\text{LDL}^\top$ for the SPD LKF innovation covariance, LU pivoting for the potentially non-SPD EKF innovation covariance, Joseph-form covariance updates, heap allocation for all large matrices, and the A&S minimax polynomial for `atan2` — together produce a numerically stable and physically valid implementation.